

Relative path: run_streaming_demo.py

Filename: run_streaming_demo.py Extension: .py

"""

OASIS – lightweight streaming feasibility demo

=====

Not a real message-queue deployment (see report Section 9 for what a production build would add – Kafka/MQTT, external per-entity state store). This demonstrates the actual claim made there: because scoring is stateless per-session, a session can be scored the moment it arrives, one at a time, rather than only in a single batch pass. Replays sessions.jsonl in timestamp order with a short simulated arrival gap, scoring each session individually and logging per-session latency.

"""

```
import json, time
from pathlib import Path
from detection.model import DetectionLayer
```

```
DATA_DIR = Path(__file__).resolve().parent / "data"
RESULTS_DIR = Path(__file__).resolve().parent / "results"
ARRIVAL_DELAY_SEC = 0.05
FLAG_THRESHOLD = 0.55 # match whatever Section 8 settles on
```

```
def main():
    sessions = [json.loads(l) for l in open(DATA_DIR / "sessions.jsonl")]
    sessions.sort(key=lambda s: s["start_time"])
    ground_truth = json.load(open(DATA_DIR / "ground_truth.json"))
    normal_sessions = [s for s in sessions if ground_truth[s["session_id"]]["label"] == "normal"]
```

```
    print("Fitting detector once, as a production deployment would at startup...")
    detector = DetectionLayer(cold_start_min_sessions=5, embed_dim=8, hidden_dim=16, max_len=15)
    detector.fit(normal_sessions, epochs=4, lr=0.02, verbose=False)
```

```
    print(f"\nReplaying {len(sessions)} sessions as a live stream "
          f"({ARRIVAL_DELAY_SEC}s simulated arrival gap)...")
```

```
    latencies, alert_count = [], 0
    with open(RESULTS_DIR / "streaming_demo_log.txt", "w") as log:
        for session in sessions:
            t0 = time.time()
            ev = detector.score_all([session])[0]
            latency_ms = (time.time() - t0) * 1000
            latencies.append(latency_ms)
            line = (f"[{session['start_time']}] {session['session_id']} "
                   f"entity={session['entity_id']:10s} anomaly_score={ev['anomaly_score']:.3f} "
                   f"scored_in={latency_ms:.1f}ms")
            if ev["anomaly_score"] >= FLAG_THRESHOLD:
                alert_count += 1
                line += " -> ALERT"
            print(line); log.write(line + "\n")
            time.sleep(ARRIVAL_DELAY_SEC)
```

```
    print(f"\nStreamed {len(sessions)} sessions, {alert_count} alerts raised.")
    print(f"Mean per-session scoring latency: {sum(latencies)/len(latencies):.2f}ms "
          f"(max {max(latencies):.1f}ms).")
```

```
if __name__ == "__main__":
    main()
```